

SIH26155 Project Resource & Dataset Guide

AI-Driven Multi-Vendor Network Security Compliance Auditor

Description, purpose, and project usefulness of the security resources, datasets, repositories, and tools collected so far.

Prepared for the SIH project team

1. Project Overview

The project audits network-device configurations against security requirements. Its intended workflow is: parse configuration, normalize it into a Common Security Model (CSM), evaluate deterministic rules, report Pass/Fail/Unknown, use AI to interpret unmapped CLI lines, obtain human approval, generate approved remediation, check for conflicts, and produce an auditable report.

AI should assist with interpretation and mapping; it should not independently make the final compliance decision or execute configuration changes.

2. Collected Resources and Their Use

Resource	What it is	Use in this project	Type
CIS Cisco Benchmark	Cisco-specific security hardening benchmark containing recommended settings, audit checks, and remediation guidance.	Primary source for the first deterministic compliance rules and remediation explanations.	Core rule source
DISA Cisco IOS-XE STIG	Government security guidance for Cisco IOS-XE routers and switches with rule IDs, severity, checks, fixes, and mappings.	Adds government-grade checks, severity, and a second source for cross-validation.	Core rule source
NIST SP 800-53 / OSCAL	Security and privacy control catalog; OSCAL provides machine-readable content formats.	Maps detailed CIS/STIG findings to broader security controls and supports structured reports.	Control reference
ISO/IEC 27001	Information-security management-system standard focused on governance, risk, and organizational controls.	Useful for high-level compliance and risk reporting; not a direct Cisco CLI rule source.	Optional reference
NetworkConfigPro	Repository containing multi-vendor network configuration examples, test data, and configuration variations.	Supplies raw Cisco configuration examples for parsing, evaluation, and regression testing.	Configuration resource
Klevernet Cisco Config Parser	Software parser for Cisco configuration syntax across Cisco platforms.	Converts raw hierarchical configuration text into structured data before CSM normalization.	Parsing tool
Cisco Running Config Validator	Reference project showing condition-based checks against Cisco running configurations.	Helps the team design rule checks and understand validation output.	Implementation reference
Lightweight DISA STIG Scanner	Offline scanner using structured rules, sample configurations, and compliance-style outputs.	Useful for YAML rule design, scanner logic, and report structure; not the authoritative benchmark itself.	Implementation reference

3. What These Resources Actually Represent

Standards and benchmarks: CIS, DISA STIG, NIST, and ISO provide requirements, controls, mappings, and guidance. They are reference sources, not complete machine-learning datasets.

Configuration resources: NetworkConfigPro and Cisco examples provide raw configuration text that can be parsed and tested.

Software repositories: Parser and scanner repositories are tools or implementation references. They help build the system but are not training datasets by themselves.

Project-specific datasets: The team must create labelled configurations, structured rules, AI mappings, reviewer feedback, remediation pairs, and test cases to satisfy the exact project workflow.

4. Datasets Required for the Complete Prototype

Dataset	Contents	Suggested MVP size	Purpose
Structured compliance rules	Rule ID, source/version, platform, severity, description, check logic, Pass/Fail/Unknown conditions, remediation, NIST mapping.	10–15 rules	Drives deterministic and repeatable evaluation.
Configuration dataset	Sanitized Cisco IOS/IOS-XE configurations categorized as secure, insecure, mixed, or unknown.	20–50 files	Tests parser and evaluator behavior.
Configuration labels	Config ID, rule ID, expected status, reason, source, and confidence.	Per rule/config pair	Provides ground truth for evaluation.
Unmapped CLI dataset	CLI line, context, platform, expected meaning, security category, CSM mapping, suggested rule, confidence.	50–100 samples	Supports AI interpretation of unfamiliar commands.
Reviewer feedback	AI suggestion, confidence, approved/rejected/corrected action, final mapping, comments, timestamp.	20–50 reviews	Demonstrates human-in-the-loop governance.
Remediation dataset	Before config, finding, approved fix, after config, template, conflict status, approval requirement.	15–30 examples	Tests safe remediation generation.
Test-case dataset	Input, target rule, expected status, expected finding, expected remediation, edge case.	30–50 tests	Proves correctness and prevents regressions.
Audit/report examples	Findings, evidence, source rule, reviewer decision, remediation, hash-chain entry, report metadata.	Several complete runs	Demonstrates traceability and integrity.

5. Recommended Folder Structure

project-data/

- benchmarks/ (CIS and DISA STIG files)
- frameworks/ (NIST OSCAL and ISO references)
- configurations/ (raw, secure, insecure, mixed)
- rules/ (structured rules and CSM mappings)
- unmapped_cli/ (CLI samples and AI interpretations)
- reviews/ (approval, rejection, correction records)
- remediation/ (before/after pairs and Jinja2 templates)
- test_cases/ (inputs and expected outputs)
- reports/ (audit reports and verification evidence)

6. How the Resources Fit Together

- 1 Extract and verify a small set of CIS and DISA STIG requirements.
- 2 Convert each requirement into a structured rule with explicit check, pass, fail, and unknown logic.
- 3 Parse Cisco configuration samples using the selected parser.
- 4 Normalize parsed data into the Common Security Model.
- 5 Run deterministic rules and compare results with expected labels.
- 6 Send unmapped CLI lines to the AI interpretation component.
- 7 Store AI explanation, confidence, suggested mapping, and evidence.
- 8 Ask a human reviewer to approve, reject, or correct the suggestion.
- 9 Generate remediation only from approved templates and run conflict checks.
- 10 Create a report and hash-chained audit log showing every decision.

7. Data Quality and Safety Notes

Keep the original source, version, license, extraction date, and rule identifier for every imported item. Community repositories may contain examples or custom rules; do not describe them as official CIS or DISA rules without verification.

Sanitize all configurations. Never store real passwords, private keys, tokens, SNMP communities, or sensitive production addresses in the dataset.

8. Recommended MVP Target

A practical MVP target is 10–15 structured rules, 20–50 sanitized Cisco configurations, 50–100 unmapped CLI samples, 20–50 reviewer decisions, 15–30 remediation examples, and 30–50 automated test cases. This demonstrates the complete workflow without claiming to train a large general-purpose AI model.

9. Final Summary

The resources collected so far are sufficient to begin implementation. CIS and DISA STIG should be the primary rule foundation; NIST OSCAL should support control mapping; configuration repositories should provide raw examples; and parser/scanner repositories should guide implementation. The remaining work is to convert these references into a verified, project-specific dataset and connect it to the deterministic engine, AI mapping module, reviewer workflow, remediation generator, and audit-report system.

10. Reference Links

CIS Cisco Benchmarks: <https://www.cisecurity.org/benchmark/cisco>
DISA STIG Downloads: <https://public.cyber.mil/stigs/downloads/>
NIST SP 800-53: <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>
NIST OSCAL: <https://csrc.nist.gov/Projects/Open-Security-Controls-Assessment-Language>
NIST OSCAL Content: <https://github.com/usnistgov/oscal-content>
ISO/IEC 27001: <https://www.iso.org/standard/27001>

NetworkConfigPro: <https://github.com/E-Conners-Lab/NetworkConfigPro>

Cisco Config Parser: https://github.com/Klevertet/cisco_config_parser

Cisco Running Config Validator: <https://github.com/iustinam/cisco-running-config-validator>

Lightweight STIG Scanner: <https://github.com/nickrusso42518/stig>